

Explanation of Methods and Functions Used in Titanic Data Analysis

1. Loading the Dataset

- `pd.read_csv('titanic.csv')` reads the Titanic dataset from a CSV file into a pandas DataFrame.

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv('titanic.csv')
```

2. Title Extraction and Survival Rate

- `str.extract()` extracts substrings using a regular expression.
- `groupby('Title')['Survived'].mean()` groups by Title and computes the survival rate.

Explanation of the Code:

```
df['Title'] = df['Name'].str.extract('([A-Za-z]+)\.', expand=False)
```

This extracts titles such as Mr, Mrs, Miss, etc. from the Name field using regex. The regex `([A-Za-z]+)` looks for a sequence of letters followed by a period (dot). The result is stored in a new column called 'Title'. The `expand=False` ensures the result is a Series.

```
1 df['Title'] = df['Name'].str.extract('([A-Za-z]+)\.', expand=False)
2 title_survival = df.groupby('Title')['Survived'].mean()
3 print(title_survival)
```

3. Cabin Grouping and Survival Rate

- `fillna('Unknown')` replaces missing cabin values.
- `astype(str).str[0]` extracts the first letter of the Cabin.

```
1 df['CabinGroup'] = df['Cabin'].fillna('Unknown').astype(str).str[0]
2 cabin_survival = df.groupby('CabinGroup')['Survived'].agg(['mean', 'count'])
3 print(cabin_survival)
```

4. Family Size Impact

- Creates `FamilySize = SibSp + Parch + 1`.
- Categorizes the size using a custom function.

```

1 df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
2
3 def categorize_family(size):
4     if size == 1:
5         return 'Single'
6     elif 2 <= size <= 4:
7         return 'Small'
8     else:
9         return 'Large'
10
11 df['FamilyCategory'] = df['FamilySize'].apply(categorize_family)
12 family_survival = df.groupby('FamilyCategory')['Survived'].mean()
13 print(family_survival)

```

5. Age Binning and Survival Rate

- `pd.cut()` creates age bins.

```

1 bins = [0, 10, 20, 30, 40, 50, 60, 70, 100]
2 labels = ['0-10', '11-20', '21-30', '31-40', '41-50', '51-60', '61-70', '71+']
3 df['AgeGroup'] = pd.cut(df['Age'], bins=bins, labels=labels)
4 age_survival = df.groupby('AgeGroup')['Survived'].mean()
5 print(age_survival)

```

6. Cross Tab: Class vs. Sex vs. Survival

```

1 cross_tab = pd.pivot_table(df, values='Survived', index='Pclass', columns='Sex', aggfunc='
    mean')
2 print(cross_tab)

```

7. Impute Missing Age by Title

- Uses `groupby().transform()` to fill missing ages.

```

1 df['Age'] = df.groupby('Title')['Age'].transform(lambda x: x.fillna(x.median()))

```

8. Correlation Heatmap

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 df_encoded = df.copy()
5 df_encoded['Sex'] = df_encoded['Sex'].map({'male': 0, 'female': 1})
6 df_encoded['Embarked'] = df_encoded['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
7 df_encoded['Title'] = df_encoded['Title'].astype('category').cat.codes
8
9 corr = df_encoded[['Survived', 'Pclass', 'Sex', 'Age', 'Fare', 'Embarked', 'Title']].corr()
10 sns.heatmap(corr, annot=True, cmap='coolwarm')
11 plt.show()

```

9. Duplicate Ticket Holders

```

1 ticket_groups = df.groupby('Ticket')
2 ticket_survival = ticket_groups['Survived'].agg(['mean', 'count']).reset_index()
3 duplicates = ticket_survival[ticket_survival['count'] > 1]
4 print(duplicates)

```

10. Fare per Person and Correlation

```
1 df['FarePerPerson'] = df['Fare'] / df['FamilySize']
2 correlation = df[['FarePerPerson', 'Survived']].corr()
3 print(correlation)
```

11. Feature Interaction: Class and Fare Category

```
1 df['FareBand'] = pd.qcut(df['Fare'], 4, labels=['1_LowFare', '2_MedLow', '3_MedHigh', '4_HighFare'])
2 df['ClassFare'] = df['Pclass'].astype(str) + '_' + df['FareBand'].astype(str)
3 classfare_survival = df.groupby('ClassFare')['Survived'].mean()
4 print(classfare_survival)
```